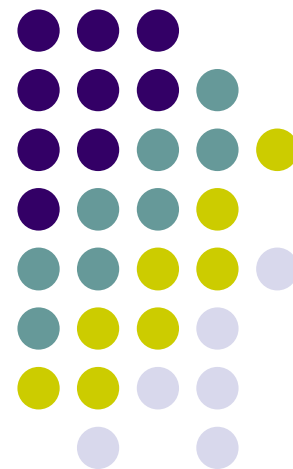
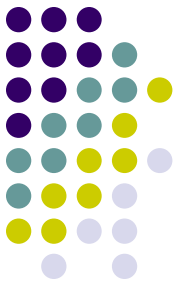


第7章 概率算法

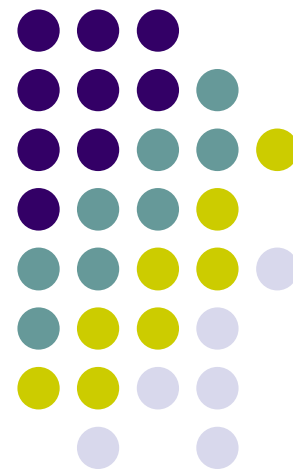


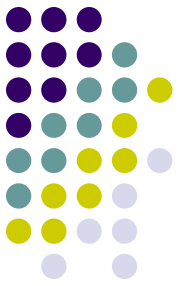


学习要点

- 理解产生伪随机数的算法
- 掌握数值随机化算法的设计思想
- 掌握舍伍德算法的设计思想
- 掌握拉斯维加斯算法的设计思想
- 掌握蒙特卡罗算法的设计思想

伪随机数





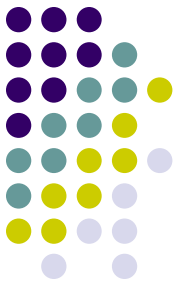
随机数

随机数在随机化算法设计中扮演着十分重要的角色。在现实计算机上无法产生真正的随机数，因此在随机化算法中使用的随机数都是一定程度上随机的，即伪随机数。

线性同余法是产生伪随机数的最常用的方法。由线性同余法产生的随机序列 $a_0, a_1, \dots, a_n$ 满足

$$\begin{cases} a_0 = d \\ a_n = (ba_{n-1} + c) \bmod m \end{cases} \quad n = 1, 2, \dots$$

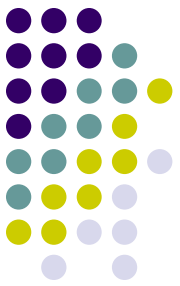
设 m 是一个给定的正整数，如果两个整数 a ， b 用 m 除，所得的余数相同，则称 a ， b 对模 m 同余。



线性同余法

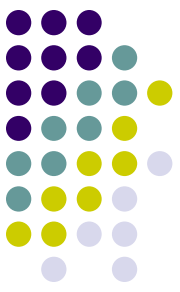
$$\begin{cases} a_0 = d \\ a_n = (ba_{n-1} + c) \bmod m \end{cases} \quad n = 1, 2, \dots$$

其中 $b \geq 0$ ， $c \geq 0$ ， $d \leq m$ 。 d 称为该随机序列的种子。如何选取该方法中的常数 b 、 c 和 m 直接关系到所产生的随机序列的随机性能。这是随机性理论研究的内容，已超出本书讨论的范围。从直观上看， m 应取得充分大，因此可取 m 为机器大数，另外应取 $\gcd(m, b) = 1$ ，因此可取 b 为一素数。



$$X[i+1] = (A * X[i] + C) \bmod M$$

- 经前人研究表明，在 $M=2^q$ 的条件下，参数 A ， C ， $X[0]$ 按如下选取，周期较大，概率统计特性好：
- $A=2^b+1=2^{(\log_2(M)/2)+1}=2^{\log_2(\sqrt{M})+1}=\sqrt{M}+1$ ； b 取 $q/2$ 附近的数
- $C=(1/2+\sqrt{3})*M$
- $X[0]$ 为任意非负数
- M ，模数 $0 < M$
- A ，乘数 $0 \leq A < M$
- C ，增量 $0 \leq C < M$
- X_i ，开始值 $0 \leq X_i < M$
- 随机数的生成在某一周期内成线性增长的趋势



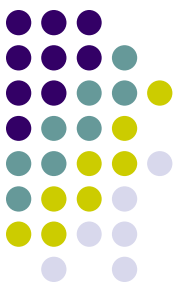
$$X[i+1]=(A*X[i]+C) \bmod M$$

- 同余序列总是进入一个循环，这是一个事实，它最终必定在N个数之间无休止的重复循环。
- 使用该方法产生的伪随机数能不能近似真正的随机效果，跟四个整数的设置相关：
 1. 序列的开始值，一般取正整数。
 2. m: 一个同余序列的周期不可能多于m个元素，所以，为了达到预期的随机效果，一般我们希望这个值稍稍大一点。在大多数情况下，当 $m = 2^e$ （e表示计算机的字大小）时，在计算机中得到的随机效果就比较令人满意了。而且这样对于随机数生成速度也是比较合理的。



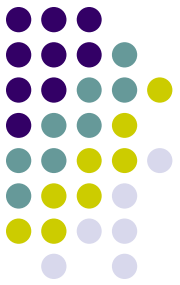
$$X[i+1] = (A * X[i] + C) \bmod M$$

3. a: 当 $a=1$ 的时候, $X_n = (X_0 + nc) \bmod m$, 它不具有随机序列的特性; 而当 $a=0$ 的时候甚至更糟糕。因此, 为实用起见, 选择 $2 \leq a < m$ 比较合理。
4. c: 当 $c=0$ 时, 数的生成过程比 $c \neq 0$ 的时候要稍微快些, 它的限制缩短了序列的周期长度, 但是也仍然有可能得到一个相当长的周期。当 $c=0$ 时被称为乘同余法, $c \neq 0$ 称为混合同余法。



$$X[i+1] = (A * X[i] + C) \bmod M$$

- 由 m ， a ， c 和 X_0 所定义的线形同余序列得到最大的周期长度 m 的条件如下：
当且仅当
 - (1) c 与 m 互素。
 - (2) 对于整除 m 的每个素数 p ， $2^b - 1$ 是 p 的倍数。
 - (3) 如果 m 是4的倍数，则 b 也是4的倍数。



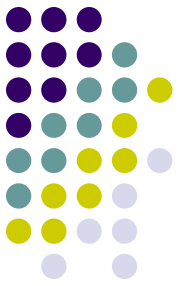
获取系统时间

- `#include<ctime>`
- `time_t now_time;`
`now_time = time(NULL);`
- `random`
- `frandom`



- `const unsigned long maxshort=65536L;`
- `const unsigned long multiplier=1194211693L;`
- `const unsigned long adder=12345L;`
- `class RandomNumber`
- `{ private:`
- `//当前种子`
- `unsigned long randseed;`
- `public:`
- `//构造函数，缺省值0表示由系统自动产生种子`
- `RandomNumber(unsigned long s=0);`
- `//产生0-n-1之间的随机整数`
- `unsigned short Random(unsigned long n);`
- `//产生[0, 1)之间的随机实数`
- `double fRandom(void);`
- `};`

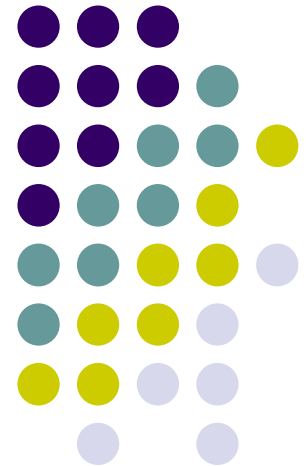
- `RandomNumber::RandomNumber(unsigned long s)`
- `{ if(s==0)`
- `randseed=time(0);`
- `else`
- `randseed=s;`
- `}`
- `unsigned short RandomNumber::Random(unsigned long n)`
- `{ randseed=multiplier*randseed+adder;`
- `return (unsigned short)((randseed>>16)%n);`
- `}`
- `double RandomNumber::fRandom(void)`
- `{ return`
- `Random(maxshort)/double(maxshort);`
- `}`

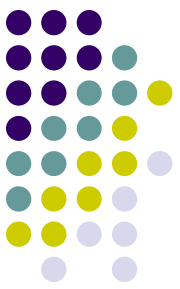


模拟抛硬币实验

- 假设抛10次硬币，每次得到正反面都是随机
- 记录50000次模拟
- `head[i]`记录得到*i*次正面的次数
- 图7-1，正态分布图
- P243

数值随机化算法





数值概率算法

- 数值概率算法常用于数值问题的求解。
- 这类算法所得到的往往是近似解，且近似解的精度随计算时间的增加而不断提高。
- 在许多情况下，要计算出问题的精确解是不可能的或没有必要，因此用数值概率算法可得到相当满意的解。

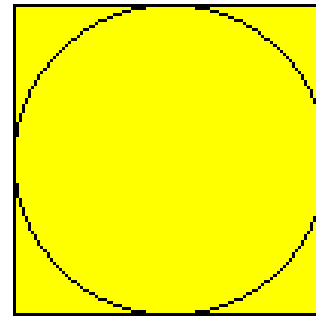


用随机投点法计算 π 值

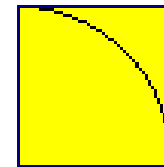
设有一半径为 r 的圆及其外切四边形。向该正方形随机地投掷 n 个点。设落入圆内的点数为 k 。由于所投入的点在正方形上均匀分布，因而所投入的点落入圆内的概率为 $\frac{\pi r^2}{4r^2} = \frac{\pi}{4}$ 。所以当 n 足够大

时， k 与 n 之比就逼近这一概率。从而 $\pi \approx \frac{4k}{n}$

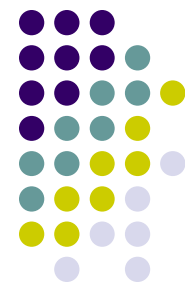
```
double Darts(int n)
{ // 用随机投点法计算 $\pi$ 值
  static RandomNumber dart;
  int k=0;
  for (int i=1;i <=n;i++) {
    double x=dart.fRandom();
    double y=dart.fRandom();
    if ((x*x+y*y)<=1) k++;
  }
  return 4*k/double(n);
}
```



(a)



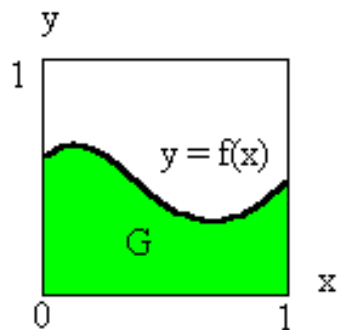
(b)



计算定积分

设 $f(x)$ 是 $[0, 1]$ 上的连续函数，且 $0 \leq f(x) \leq 1$ 。

需要计算的积分为 $I = \int_0^1 f(x)dx$ ，积分 I 等于图中的面积 G 。

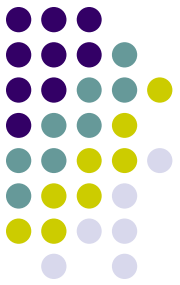


在图所示单位正方形内均匀地作投点试验，则随机点落在曲线下方的概率为

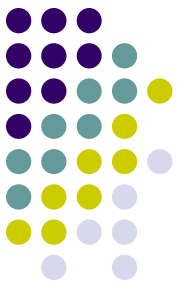
$$P_r \{y \leq f(x)\} = \int_0^1 \int_0^{f(x)} dy dx = \int_0^1 f(x) dx$$

假设向单位正方形内随机地投入 n 个点 (x_i, y_i) 。如果有 m 个点落入

G 内，则随机点落入 G 内的概率 $I \approx \frac{m}{n}$



```
public static double darts1(int n)
{
    int k=0;
    for(int i=1;i<=n;i++){
        double x=dart.fRandom();
        double y=dart.fRandom();
        if(y<=f(x)) k++;
    }
    return k/(double)n;
}
```



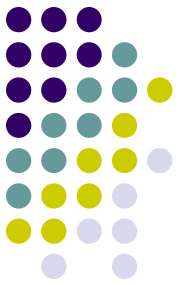
$$I = \int_a^b f(x)dx$$

- 被积函数 $f(x)$ 在区间 $[a,b]$ 中有界，并用 M ， L 分别表示其最大值和最小值。此时可进行变量代换 $x=a+(b-a)z$ ，将所求积分变为 $I = cI^* + d$ ，其中：

$$c = (M - L)(b - a), d = L(b - a), I^* = \int_0^1 f^*(z)dz$$

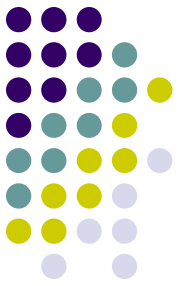
$$f^*(z) = \frac{1}{M - L} [f(a + (b - a)z) - L], \text{ 且有 } 0 \leq f^*(z) \leq 1$$

- 因此， I^* 可用随机投点法计算。



课堂作业--课后习题1

```
double RandomNumber::Norm()
{
    double s,t,p,q;
    while(true){
        s=2*fRandom()-1;
        t=2*fRandom()-1;
        p=s*s+t*t;
        if(p<1) break;
    }
    q=sqrt((-2*log(p))/p);
    return s*q;
}
```



扩展到一般的正态分布的算法

```
double RandomNumber::Norm(double a ,double b)
{
    double x=Norm();
    return a+b*x;
}
```

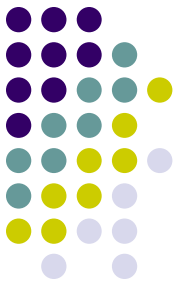


用平均值法计算定积分

平均值法：为计算定积分 $J = \int_a^b f(x)dx$ ，设随机变量 X 服从 (a,b) 上的

均匀分布，则 $X=f(X)$ 的数学期望为. $E(f(X)) = \int_a^b f(x)dx = J$. 由辛钦大数定律，

可以得到 $J \approx \frac{b-a}{n} \sum_{i=1}^n f(x_i)$ 。



- 在 $[a,b]$ 区间上随机抽取 n 个点 $x_i(i=1,2,\dots,n)$, 则均值

$$\bar{I} = \frac{b-a}{n} \sum_{i=1}^n g(x_i)$$

可作为所求积分 I 的近似值。

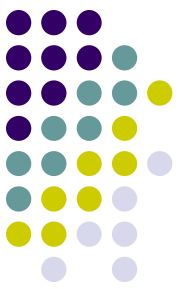
```
public static double integration(double a, double b, int n)
{
    //用平均值法计算定积分
    Random rnd = new Random();
    double y=0;
    for(int i=1; i<=n; i++){
        double x=(b-a)*rnd.fRandom()+a;
        y+=f(x);
    }
    return (b-a)*y/(double)n;
}
```



求解下面的非线性方程组

$$\begin{cases} f_1(x_1, x_2, \dots, x_n) = 0 \\ f_2(x_1, x_2, \dots, x_n) = 0 \\ \dots\dots\dots \\ f_n(x_1, x_2, \dots, x_n) = 0 \end{cases}$$

其中, $x_1, x_2, \dots, x_n$ 是实变量, f_i 是未知量 $x_1, x_2, \dots, x_n$ 的非线性实函数。要求确定上述方程组在指定求根范围内的一组解 $x_1^*, x_2^*, \dots, x_n^*$



基本思想

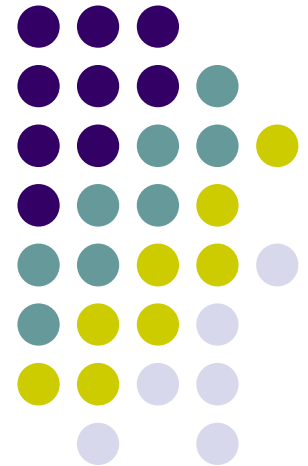
- 构造函数 $\phi(x) = \sum_{i=1}^n f_i^2(x)$ ，其中 $x = (x_1, x_2, \dots, x_n)$ 。
- 易知，该函数的零点即是所求非线性方程组的一组解。
- 简单随机模拟算法：
 1. 在指定求根区域内，选定一个 x_0 作为根的初值。按照预先选定的分布，逐个选取随机点 x 。
 2. 计算目标函数 $\phi(x)$ ，并把满足精度要求的随机点 x 作为所求非线性方程组的近似解。
- 缺点：工作量大

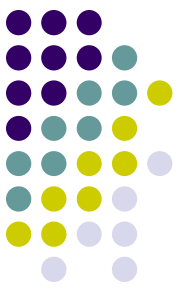


在指定求根区域 D 内，选定一个随机点 x_0 作为随机搜索的出发点。在算法的搜索过程中，假设第 j 步随机搜索得到的随机搜索点为 x_j 。在第 $j+1$ 步，计算出下一步的随机搜索增量 Δx_j 。从当前点 x_j 依 Δx_j 得到第 $j+1$ 步的随机搜索点。当 $\phi(x_{j+1}) < \varepsilon$ 时，取为所求非线性方程组的近似解。否则进行下一步新的随机搜索过程。

解读P248程序

舍伍德 (Sherwood) 算法





舍伍德 (Sherwood) 算法

- 舍伍德算法总能求得问题的一个解，且所求得的解总是正确的。
- 当一个确定性算法在最坏情况下的计算复杂性与其在平均情况下的计算复杂性有较大差别时，可以在这个确定算法中引入随机性将它改造成一个舍伍德算法，消除或减少问题的好坏实例间的这种差别。
- 舍伍德算法精髓不是避免算法的最坏情况行为，而是设法消除这种最坏行为与特定实例之间的关联性。

舍伍德 (Sherwood) 算法



设 A 是一个确定性算法，当它的输入实例为 x 时所需的计算时间记为 $t_{A(x)}$ 。设 X_n 是算法 A 的输入规模为 n 的实例的全体，则当问题的输入规模为 n 时，算法 A 所需的平均时间为

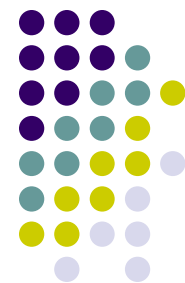
$$\bar{t}_A(n) = \sum_{x \in X_n} t_A(x) / |X_n|$$

这显然不能排除存在 $x \in X_n$ 使得 $t_A(x) \gg \bar{t}_A(n)$ 的可能性。希望获得一个随机化算法 B ，使得对问题的输入规模为 n 的每一个实例均有

$$t_B(x) = \bar{t}_A(n) + s(n)$$

这就是舍伍德算法设计的基本思想。当 $s(n)$ 与 $t_{A(n)}$ 相比可忽略时，舍伍德算法可获得很好的平均性能。

舍伍德 (Sherwood) 算法



复习学过的Sherwood算法:

阅 读 书 上 例 程

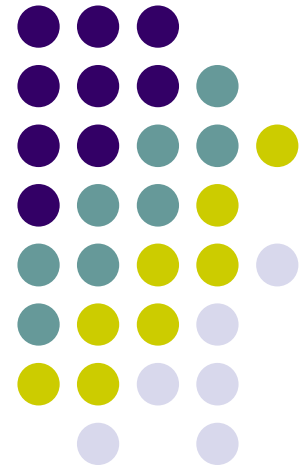
(1) 线性时间选择算法

(2) 快速排序算法

有时也会遇到这样的情况，即所给的确定性算法无法直接改造成舍伍德型算法。此时可借助于随机预处理技术，不改变原有的确定性算法，仅对其输入进行随机洗牌，同样可收到舍伍德算法的效果。例如，对于确定性选择算法，可以用下面的洗牌算法**shuffle**将数组**a**中元素随机排列，然后用确定性选择算法求解。这样做所收到的效果与舍伍德型算法的效果是一样的。

```
template<class Type>
void Shuffle(Type a[], int n)
{ // 随机洗牌算法
    static RandomNumber rnd;
    for (int i=0; i<n; i++) {
        int j=rnd.Random(n-i)+i;
        Swap(a[i], a[j]);
    }
}
```

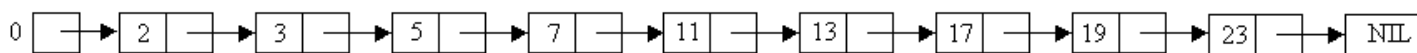
舍伍德 (Sherwood) 算法



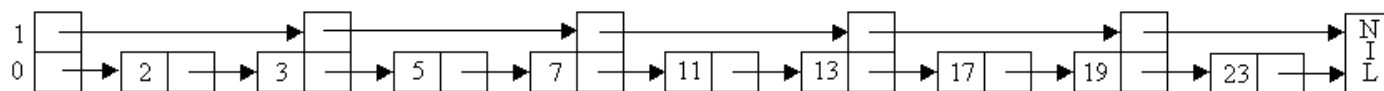
跳跃表



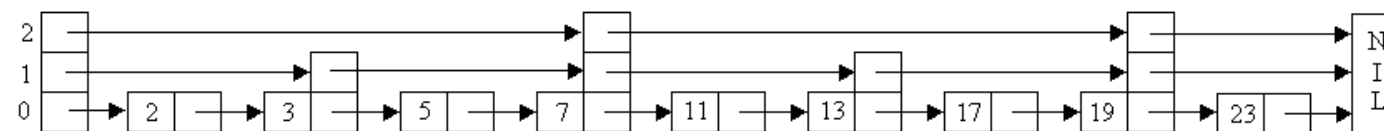
- 舍伍德型算法的设计思想还可用于设计高效的数据结构。
- 如果用有序链表来表示一个含有 n 个元素的有序集 S ，则在最坏情况下，搜索 S 中一个元素需要 $\Omega(n)$ 计算时间。
- 提高有序链表效率的一个技巧是在有序链表的部分结点处增设附加指针以提高其搜索性能。在增设附加指针的有序链表中搜索一个元素时，可借助于附加指针跳过链表中若干结点，加快搜索速度。这种增加了向前附加指针的有序链表称为**跳跃表**。



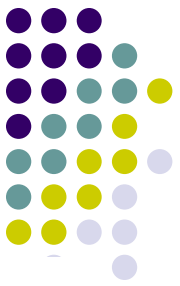
(a)



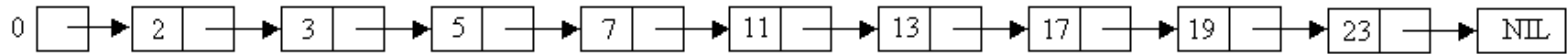
(b)



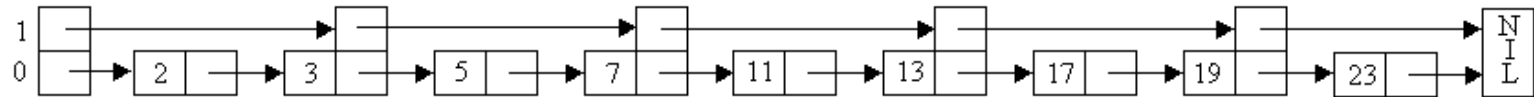
(c)



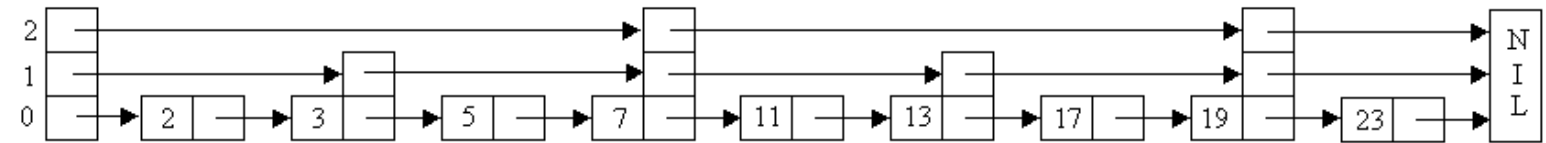
跳跃表



(a)



(b)



(c)

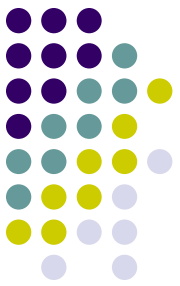
图a: 没有附加指针的有序链表

图b: 在图a的基础上增加了跳跃一个结点的附加指针

图c: 在图b的基础上增加了跳跃3个点的附加指针

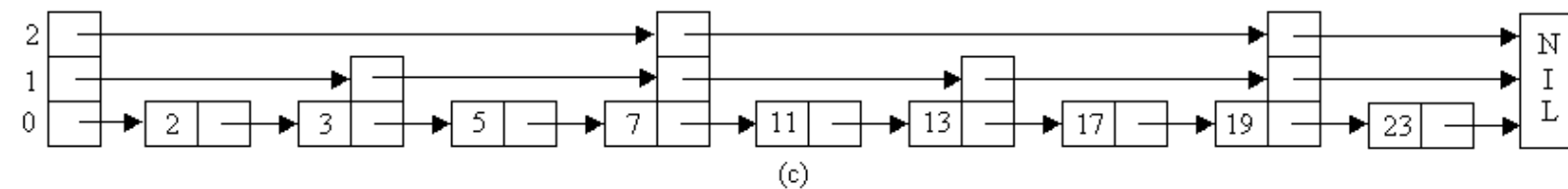
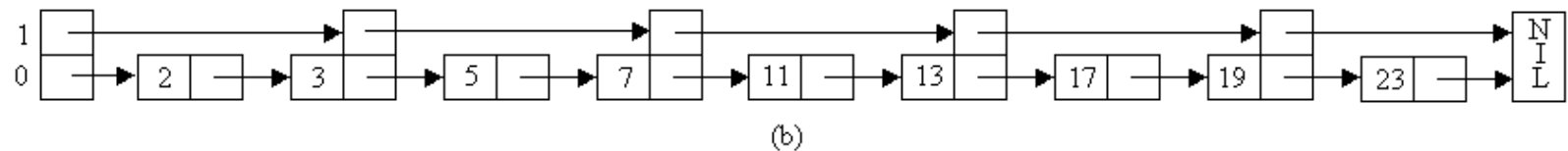
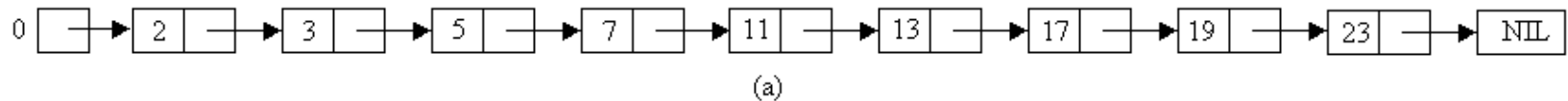
K级结点: 一个有 $k+1$ 个指针的结点

在c图中搜索元素8



跳跃表

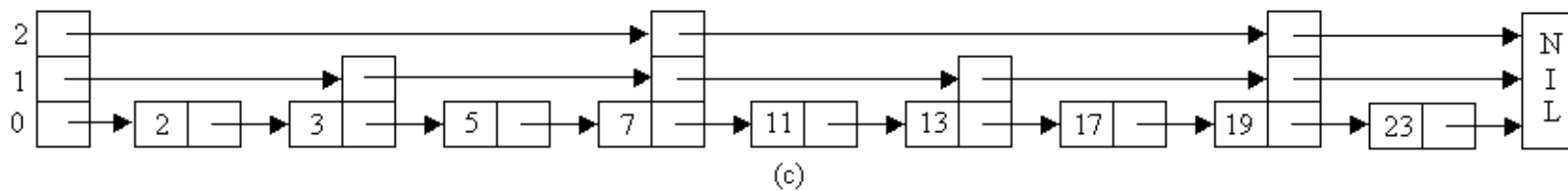
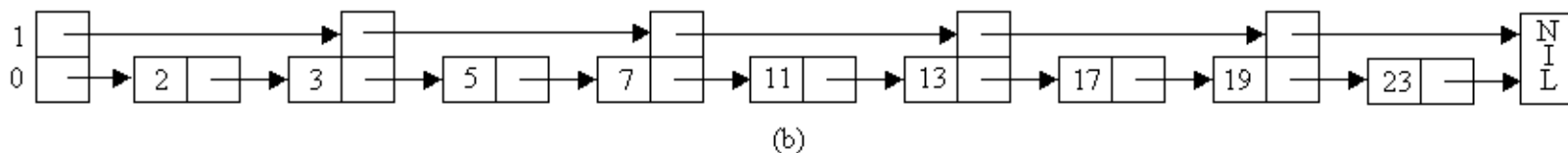
- 应在跳跃表的哪些结点增加附加指针以及在该结点处应增加多少指针完全采用**随机化方法**来确定。这使得跳跃表可在 $O(\log n)$ 平均时间内支持关于有序集搜索、插入和删除等运算。





跳跃表

在一般情况下，给定一个含有 n 个元素的有序链表，可以将它改造成一个完全跳跃表，使得每一个 k 级结点含有 $k+1$ 个指针，分别跳过 $2^k-1, 2^{k-1}-1, \dots, 2^0-1$ 个中间结点。第 i 个 k 级结点安排在跳跃表的位置 $i2^k$ 处， $i \geq 0$ 。这样就可以在时间 $O(\log n)$ 内完成集合成员的搜索运算。在一个完全跳跃表中，最高级的结点是 $\lceil \log n \rceil$ 级结点。

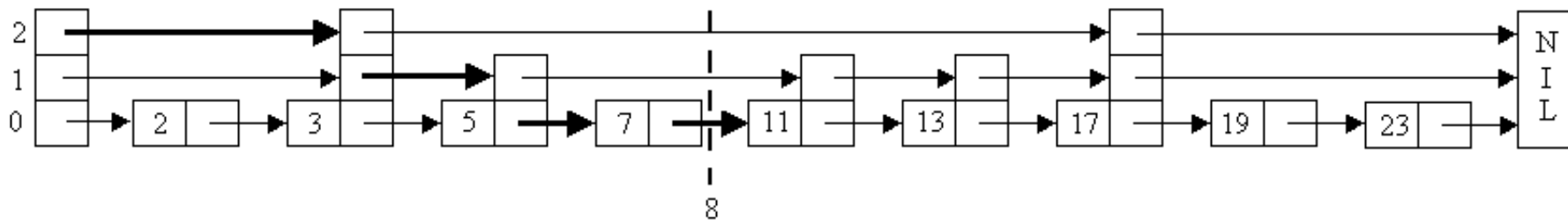


完全跳跃表与完全二叉搜索树的情形非常类似。它虽然可以有效地支持成员搜索运算，但不适应于集合动态变化的情况。集合元素的插入和删除运算会破坏完全跳跃表原有的平衡状态，影响后继元素搜索的效率。

跳跃表



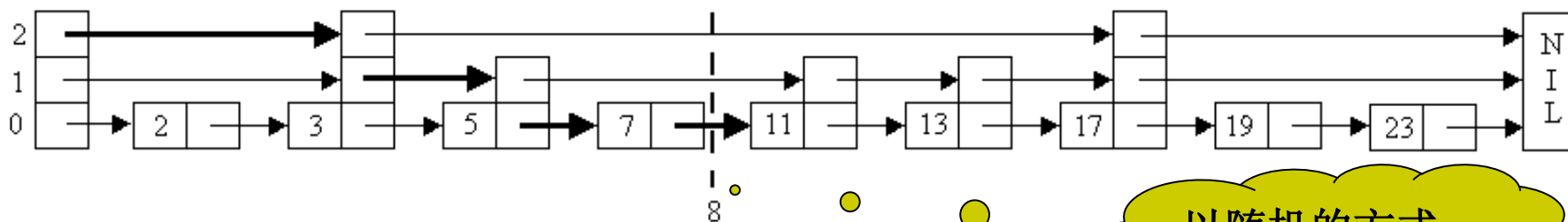
- 为了在动态变化中维持跳跃表中附加指针的平衡性，必须使跳跃表中 k 级结点数维持在总结点数的一定比例范围内。
- 注意到在一个完全跳跃表中，50%的指针是0级指针；25%的指针是1级指针；...； $(100/2^{k+1})\%$ 的指针是 k 级指针。
- 因此，在插入一个元素时，以概率 $1/2$ 引入一个0级结点，以概率 $1/4$ 引入一个1级结点，...，以概率 $1/2^{k+1}$ 引入一个 k 级结点。
- 另一方面，一个 i 级结点指向下一个同级或更高级的结点，它所跳过的结点数不再准确地维持在 2^{i-1} 。
- 经过这样的修改，就可以在插入或删除一个元素时，通过对跳跃表的局部修改来维持其平衡性。



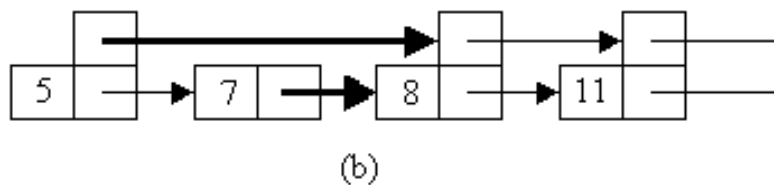
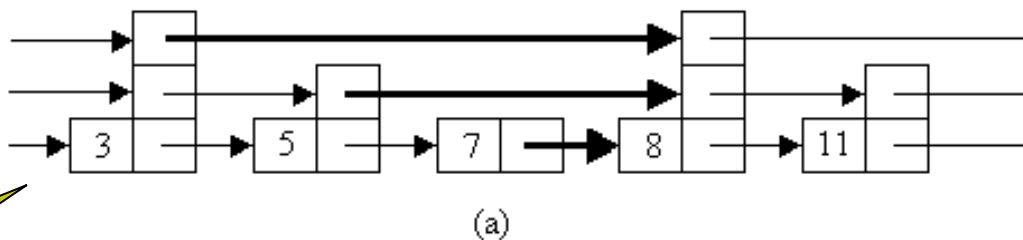
跳跃表



- 在下图所示的位置插入一个元素 8



以随机的方式
确定新结点的
级别



跳跃表

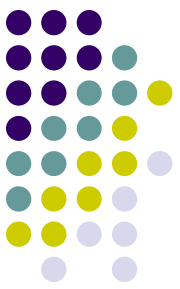


在一个完全跳跃表中，具有 i 级指针的结点中有一半同时具有 $i+1$ 级指针。为了维持跳跃表的平衡性，可以事先确定一个实数 $0 < p < 1$ ，并要求在跳跃表中维持在具有 i 级指针的结点中同时具有 $i+1$ 级指针的结点所占比例约为 p 。

为此目的，在插入一个新结点时，先将其结点级别初始化为0，然后用随机数生成器反复地产生一个 $[0, 1]$ 间的随机实数 q 。如果 $q < p$ ，则使新结点级别增加1，直至 $q \geq p$ 。

由此产生新结点级别的过程可知，所产生的新结点的级别为0的概率为 $1-p$ ，级别为1的概率为 $p(1-p)$ ，...，级别为 i 的概率为 $p^i(1-p)$ 。

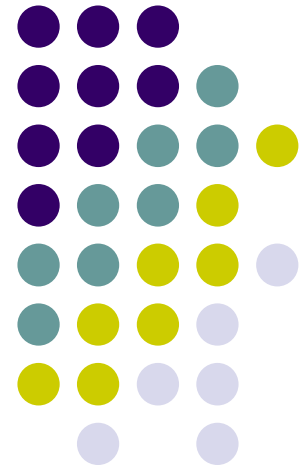
如此产生的新结点的级别有可能是一个很大的数，甚至远远超过表中元素的个数。为了避免这种情况，用 $\log_{1/p} n$ 作为新结点级别的上界。其中 n 是当前跳跃表中结点个数。当前跳跃表中任一结点的级别不超过 $\log_{1/p} n$



跳跃表的时间空间复杂性分析

- 见书P258
- 舍伍德算法的优点：
 - 计算时间复杂性对所有实例而言是相对均匀的，但与其相应的确定性算法相比，其平均时间复杂性没有改进。
- 下面看一种可以显著改进算法有效性的算法。

拉斯维加斯算法





拉斯维加斯算法

- 拉斯维加斯算法不会得到不正确的解，一旦用拉斯维加斯算法找到一个解，那么这个解肯定是正确的。
- 但是有时候用拉斯维加斯算法可能找不到解。
- 与蒙特卡罗算法类似。拉斯维加斯算法得到正确解的概率随着它用的计算时间的增加而提高。对于所求解问题的任一实例，用同一拉斯维加斯算法反复对该实例求解足够多次，可使求解失效的概率任意小。

拉斯维加斯 (Las Vegas) 算法



拉斯维加斯算法的一个显著特征是它所作的随机性决策有可能导致算法找不到所需的解。

用一个boolean型方法表示。

```
void obstinate(Object x, Object y)
{ // 反复调用拉斯维加斯算法LV(x,y), 直到找到问题的一个解y
  bool success= false;
  while (!success) success=lv(x,y);
}
```



设 $p(x)$ 是对输入 x 调用拉斯维加斯算法获得问题的一个解的概率。一个正确的拉斯维加斯算法应该对所有输入 x 均有 $p(x) > 0$ 。

设 $t(x)$ 是算法**obstinate**找到具体实例 x 的一个解所需的平均时间， $s(x)$ 和 $e(x)$ 分别是算法对于具体实例 x 求解成功或求解失败所需的平均时间，则有：

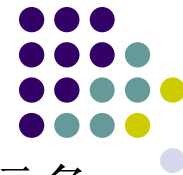
$$t(x) = p(x)s(x) + (1 - p(x))(e(x) + t(x))$$

解此方程可得：

$$t(x) = s(x) + \frac{1 - p(x)}{p(x)} e(x)$$

请解释此公式

n后问题



在 $n \times n$ 格的棋盘上放置彼此不受攻击的 n 个皇后。按照国际象棋的规则，皇后可以攻击与之处在同一行或同一列或同一斜线上的棋子。 n 后问题等价于在 $n \times n$ 格的棋盘上放置 n 个皇后，任何2个皇后不放在同一行或同一列或同一斜线上。

1				Q				
2						Q		
3								Q
4		Q						
5							Q	
6	Q							
7			Q					
8					Q			
	1	2	3	4	5	6	7	8

n后问题



对于n后问题的任何一个解而言，每一个皇后在棋盘上的位置无任何规律，不具有系统性，而更象是随机放置的。由此容易想到下面的**拉斯维加斯算法**。

在棋盘上相继的各行中随机地放置皇后，并注意使新放置的皇后与已放置的皇后互不攻击，直至n个皇后均已相容地放置好，或已没有下一个皇后的可放置位置时为止。

```
bool Queen::Place(int k)
{
    for (int j=1;j<k;j++)
        if ((abs(k-j)==abs(x[j]-x[k]))||(x[j]==x[k]))
            return false;
    return true;
}
```

方法queensLV()实现在棋盘上随机放置n个皇后的拉斯维加斯算法。见书P260。请解释

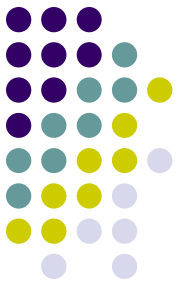


```
private static Boolean queensLV ()
{    //随机放置  $n$  个皇后的 Las Vegas 算法
    rnd=new Random ();
    int k=1;
    int count=1;
    while ((k<=n) && (count>0))    {
        count=0;
        int j=0;
        for ( int i=1; i<=n; i++ )    {
            x[k]=i;
            if (place (k))
                if (rnd.random(++count) ==0) j=i; //随机位置
        }
        if (count>0) x[k++]=j;
    }
    return (count>0); // count>0 表示放置成功
}
```



```
public static void nQueen ( )  
{ //解  $n$  皇后问题的 Las Vegas 算法  
  //初始化  $x$   
   $x = \text{new int}[n+1]$ ;  
  for( int  $i=0$ ;  $i \leq n$ ;  $i++$  )  $x[i]=0$ ;  
  // 反复调用随机放置  $n$  个皇后的 Las Vegas 算法,  
  直至放置成功  
  while( ! queensLV ( ) );  
}
```

一旦发现无法再放置下一个皇后，
就全部重新开始



如果将上述随机放置策略与回溯法相结合，可能会获得更好的效果。

可以先在棋盘的若干行中随机地放置皇后，然后在后继行中用回溯法继续放置，直至找到一个解或宣告失败。

随机放置的皇后越多，后继回溯搜索所需的时间就越少，但失败的概率也就越大。



```
private static boolean queensLV (int stopVegas)
{ //随机放置 n 个皇后 Las Vegas 算法
    rnd = new Random ();    //初始化随机数
    int k=1;                //下一个放置的皇后编号
    int count=1;
    // 1<=stopVegas<=n 表示允许随机放置的皇后数
    while ((k<=stopVegas) && (count>0)) {
        count=0;
        int j=0;
        for ( int i=1; i<=n; i++) {
            x[k]=i;
            if ( place (k))
                if ( rnd.random(++count) ==0 ) j=i;
                //随机位置
        }
        if ( count>0 ) x[k++]=j;
    }
    return ( count>0 ); //count>0 表示放置成功
}
```




```
public static void nQueen ( int stop )
{ //与回溯法相结合的解  $n$  皇后问题的 Las Vegas 算法
    x=new int[n+1];
    y=new int[n+1];
    for(int i=0; i<=n; i++ ) {
        x[i]=0;
        y[i]=0;
    }
    while (! queensLV (stop));
    //算法的回溯搜索部分
    backtrack ( stop+1 );
}
```



表 1 解 8 皇后问题的 Las Vegas 优化算法中不同的 stopVegas 值所对应的算法效率

stopVegas	p	s	e	t
0	1.0000	114.00	—	114.00
1	1.0000	39.63	—	39.63
2	0.8750	22.53	39.67	28.20
3	0.4931	13.48	15.10	29.01
4	0.2618	10.31	8.79	35.10
5	0.1624	9.33	7.29	46.92
6	0.1375	9.05	6.98	53.50
7	0.1293	9.00	6.97	55.93
8	0.1293	9.00	6.97	55.93

完全使用
回溯法



表 2 解 12 皇后问题的 Las Vegas 优化算法中不同的
stopVegas 值所对应的算法效率

stopVegas	p	s	e	t
0	1.0000	262.00	—	262.00
5	0.5039	33.88	47.23	80.39
12	0.0465	13.00	10.20	222.11

整数因子分解



设 $n > 1$ 是一个整数。关于整数 n 的因子分解问题是找出 n 的如下形式的唯一分解式： $n = p_1^{m_1} p_2^{m_2} \cdots p_k^{m_k}$

其中， $p_1 < p_2 < \dots < p_k$ 是 k 个素数， $m_1, m_2, \dots, m_k$ 是 k 个正整数。

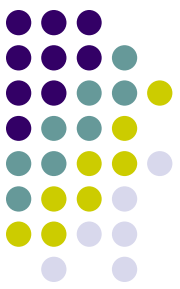
如果 n 是一个合数，则 n 必有一个非平凡因子 x ， $1 < x < n$ ，使得 x 可以整除 n 。给定一个合数 n ，求 n 的一个非平凡因子的问题称为整数 n 的因子分割问题。

```
int Split(int n)
```

```
{  
    int m = floor(sqrt(double(n)));  
    for (int i=2; i<=m; i++)  
        if (n%i==0) return i;  
    return 1;
```

算法**split(n)**是关于 m 的指数时间算法。
 m 为 n 的位数， $m = \lceil \log_{10}(1+n) \rceil$

```
} 事实上，算法split(n)是对范围在 $1 \sim x$ 的所有整数进行了试除  
而得到范围在 $1 \sim x^2$ 的任一整数的因子分割。
```



Pollard ρ 方法

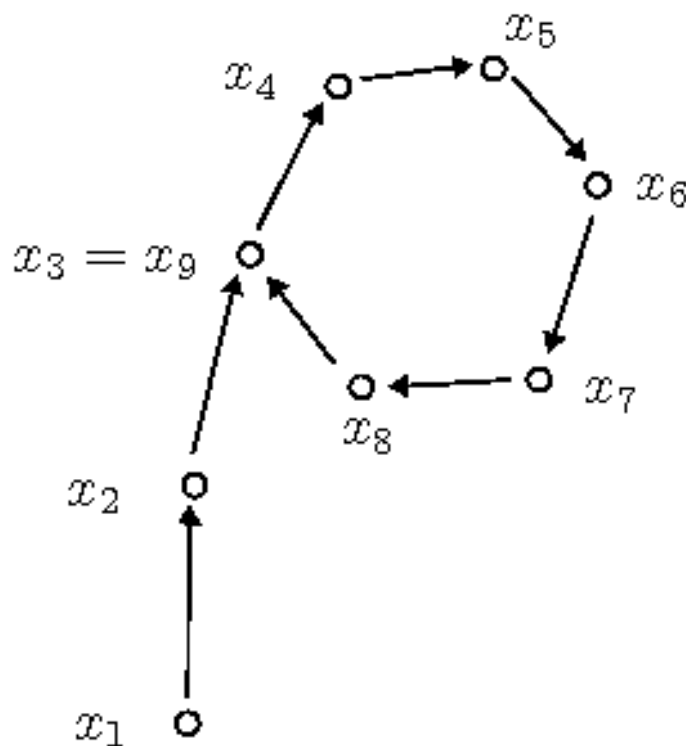
- 随机选取大约 $c\sqrt{p}$ 个整数（ c 为一个常数），就有很大概率在这些整数中找到两个是 $\text{mod } p$ 同余的。实践中可以采用同余递推序列

$$x_{i+1} \equiv f(x_i) \pmod{N}$$

来产生伪随机数。

- 设 p 是 N 的一个因子，且找到 $x_j \equiv x_i \pmod{p}$ ，则计算 $(x_j - x_i, N)$ 便可能得到 N 的一个非平凡因子。

Pollard ρ 方法



- 如上定义的一阶的递推序列 $\{x_i\}$ 在 $\text{mod } p$ 意义下必定是最终循环的(如图, 看上去就像希腊字母 ρ)。
- 具体的证明方法请参考算法导论

<http://www.csh.rit.edu/~pat/math/quirkies/rho/>

Pollard算法

算法Pollard用与split(n)相同的工作量可以得到在 $1 \sim x^4$ 范围内整数的因子分割。

在开始时选取 $0 \sim n-1$ 范围内的随机数，然后递归地由

$$x_i = (x_{i-1}^2 - 1) \bmod n$$

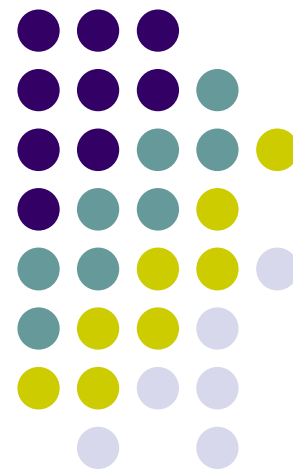
产生无穷序列 $x_1, x_2, \dots, x_k, \dots$

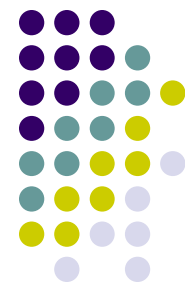
对于 $i=2^k$ ，以及 $2^k < j \leq 2^{k+1}$ ，算法计算出 $x_j - x_i$ 与 n 的最大公因子 $d = \gcd(x_j - x_i, n)$ 。如果 d 是 n 的非平凡因子，则实现对 n 的一次分割，算法输出 n 的因子 d 。

```
void Pollard(int n)
{// 求整数n因子分割的拉斯维加斯算法
    RandomNumber rnd;
    int i=1;
    int x=rnd.Random(n); // 随机整数
    int y=x;
    int k=2;
    while (true) {
        i++;
        x=(x*x-1)%n;
        int d=gcd(y-x,n); // 求n的非平凡因子
        if ((d>1) && (d<n)) cout<<d<<endl;
        if (i==k) {
            y=x;
            k*=2;}
    } }
```

对Pollard算法更深入的分析可知，执行算法的while循环约 $\sqrt{p}$ 次后，Pollard算法会输出 n 的一个因子 p 。由于 n 的最小素因子 $p \leq \sqrt{n}$ ，故Pollard算法可在 $O(n^{1/4})$ 时间内找到 n 的一个素因子。

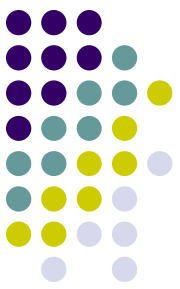
蒙特卡罗算法





蒙特卡罗算法

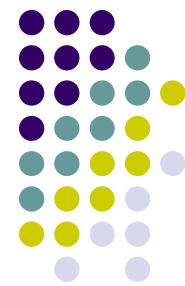
- 蒙特卡罗算法用于求问题的**准确解**。
- 对于许多问题来说，近似解毫无意义。
 - 例如，一个判定问题其解为“是”或“否”，二者必居其一，不存在任何近似解答。
 - 又如，我们要求一个整数的因子时所给出的解答必须是准确的，一个整数的近似因子没有任何意义。
- 用蒙特卡罗算法能求得问题的一个解，但这个解未必是正确的。
- 求得正确解的概率依赖于算法所用的时间。算法所用的时间越多，得到正确解的概率就越高。蒙特卡罗算法的主要缺点就在于此。一般情况下，无法有效判断得到的解是否肯定正确。



蒙特卡罗方法的基本原理及思想

- 当所要求解的问题是某种事件出现的概率，或者是某个随机变量的期望值时，它们可以通过某种“试验”的方法，得到这种事件出现的频率，或者这个随机变数的平均值，并用它们作为问题的解。
- 蒙特卡罗方法通过抓住事物运动的几何数量和几何特征，利用数学方法来加以模拟，即进行一种数字模拟实验。它是以一个概率模型为基础，按照这个模型所描绘的过程，通过模拟实验的结果，作为问题的近似解。
- 可以把蒙特卡罗解题归结为三个主要步骤：构造或描述概率过程；实现从已知概率分布抽样；建立各种估计量。

蒙特卡罗(Monte Carlo)算法



- 设 p 是一个实数，且 $1/2 < p < 1$ 。如果一个蒙特卡罗算法对于问题的任一实例得到正确解的概率不小于 p ，则称该蒙特卡罗算法是**p正确**的，且称 $p-1/2$ 是该算法的**优势**。
- 如果对于同一实例，蒙特卡罗算法不会给出2个不同的正确解答，则称该蒙特卡罗算法是**一致的**。
- 有些蒙特卡罗算法除了具有描述问题实例的输入参数外，还具有描述错误解可接受概率的参数。这类算法的计算时间复杂性通常由问题的实例规模以及错误解可接受概率的函数来描述。

蒙特卡罗(Monte Carlo)算法



对于一个一致的 p 正确蒙特卡罗算法，要提高获得正确解的概率，只要执行该算法若干次，并选择出现频次最高的解即可。

如果重复调用一个一致的 $(1/2+\varepsilon)$ 正确的蒙特卡罗算法 $2m-1$ 次，得到正确解的概率至少为 $1-\delta$ ，其中，

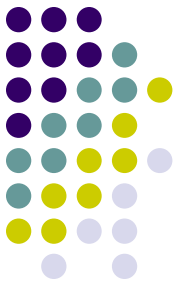
$$\delta = \frac{1}{2} - \varepsilon \sum_{i=0}^{m-1} \binom{2i}{i} \left(\frac{1}{4} - \varepsilon^2\right)^i \leq \frac{(1-4\varepsilon^2)^m}{4\varepsilon\sqrt{\pi m}}$$

对于一个解所给问题的蒙特卡罗算法 $MC(x)$ ，如果存在问题实例的子集 X 使得：

- (1) 当 $x \notin X$ 时， $MC(x)$ 返回的解是正确的；
 - (2) 当 $x \in X$ 时，正确解是 y_0 ，但 $MC(x)$ 返回的解未必是 y_0 。
- 称上述算法 $MC(x)$ 是偏 y_0 的算法。

重复调用一个一致的， p 正确偏 y_0 蒙特卡罗算法 k 次，可得到一个 $O(1-(1-p)^k)$ 正确的蒙特卡罗算法，且所得算法仍是一个一致的偏 y_0 蒙特卡罗算法。

主元素问题



- 问题：设 $A[1..n]$ 是一个含有 n 个元素的序列，若某元素在 A 中出现的次数超过了 A 中元素个数的一半，则该元素即为 A 的主元素。对于给定的序列 A ，试判断其中是否存在主元素，若存在则求之。
- 算法思想
在序列中随机选择一元素，然后统计序列中该元素的次数，若次数超过 $n/2$ ，则其为主元素，否则返回不存在主元素的信息。

主元素问题

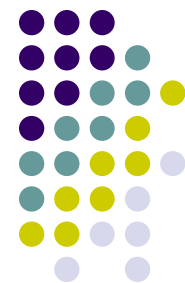


设 $T[1:n]$ 是一个含有 n 个元素的数组。当 $|\{i \mid T[i]=x\}| > n/2$ 时，称元素 x 是数组 T 的主元素。

```
template<class Type>
bool Majority(Type *T, int n)
{// 判定主元素的蒙特卡罗算法
    int i=rnd.Random(n)+1;
    Type x=T[i]; // 随机选择数组元素
    int k=0;
    for (int j=1;j<=n;j++)
        if (T[j]==x) k++;
    return (k>n/2); // k>n/2 时T含有主元素
}
```

对于任何给定的 $\epsilon > 0$ ，算法 **majorityMC** 重复调用 $\lceil \log(1/\epsilon) \rceil$ 次算法 **majority**。它是一个偏真蒙特卡罗算法，且其错误概率小于 ϵ 。算法 **majorityMC** 所需的计算时间显然是 $O(n \log(1/\epsilon))$ 。

```
template<class Type>
bool MajorityMC(Type *T, int n, double e)
{// 重复调用算法Majority
    int k=ceil(log(1/e)/log(2));
    for (int i=1;i<=k;i++)
        if (Majority(T,n)) return true;
    return false;
}
```



素数的若干结论

- **算术基本定理：**每个合数都可以唯一地表成素数的乘积。
- **素数基本定理：**设不超过 n 的素数的个数为 $\pi(n)$ ，则 $\pi(n) \sim n / \ln n$ 。
- 设 $1 < k < 2n$ ，则能整除 k 的素数的个数至多 $\pi(n)$ 个。
- **欧拉公式：**设将 n 分解为素数乘积为 $n = p_1 p_2 \dots p_r$ ，则不超过 n 且与 n 互素的正整数的个数 $j(n) = (p_1 - 1)(p_2 - 1) \dots (p_r - 1)$ 。
- **费马(Fermat)小定理（素数必要条件）：**设 p 是素数，则对于小于 p 的任意正整数 a ，均有 $a^{p-1} \equiv 1 \pmod{p}$ 。
- **梅森(Mersenne)素数：**形如 $M_p = 2^p - 1$ （ p 为素数）的素数称为梅森素数。

素数测试



Wilson定理: 对于给定的正整数 n , 判定 n 是一个素数的充要条件是 $(n-1)! \equiv -1 \pmod{n}$ 。

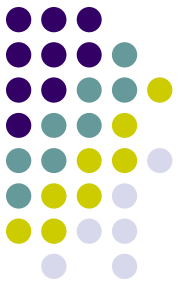
费尔马小定理: 如果 p 是一个素数, 且 $0 < a < p$, 则 $a^{p-1} \equiv 1 \pmod{p}$ 。

二次探测定理: 如果 p 是一个素数, 且 $0 < x < p$, 则方程 $x^2 \equiv 1 \pmod{p}$ 的解为 $x=1, p-1$ 。

```
void power( unsigned int a, unsigned int p,
            unsigned int n, unsigned int &result,
            bool &composite)
{
    // 计算mod n, 并实施对n的二次探测
    unsigned int x;
    if (p==0) result=1;
    else {
        power(a,p/2,n,x,composite); // 递归
        result=(x*x)%n;              // 二次探测
        if ((result==1)&&(x!=1)&&(x!=n-1))
            composite=true;
        if ((p%2)==1) // p是奇数
            result=(result*a)%n;
    }
}
```

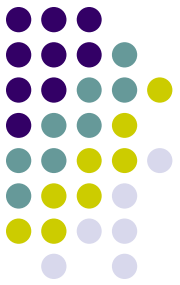
```
bool Prime(unsigned int n)
{
    // 素数测试的蒙特卡罗算法
    RandomNumber rnd;
    unsigned int a, result;
    bool composite=false;
    a=rnd.Random(n-3)+2;
}
```

算法**prime**是一个偏假3/4正确的蒙特卡罗算法。通过多次重复调用错误概率不超过 $(1/4)^k$ 。这是一个很保守的估计, 实际使用的效果要好得多。



课堂思考题

- 设一个蒙特卡洛算法能在任何情况下以至少为 $1-a$ 的概率给出正确结果。试问该算法需要重复执行多少次才能将给出正确结果的概率提高到至少 $1-b$ （其中 $0 < b < a < 1$ ）？



本周任务

- 课后习题
- 7-2、3、5
- 算法设计题：
 - 设计一个时间复杂度为 $O(n^2)$ 的算法来产生 $1, 2, \dots, n$ 的一个随机排列。要求用文字描述算法的基本思路，并分析其时间复杂度。